

MODULE CODE:	AIDL_0112
MODULE TITLE:	AIDL Course: Natural Language Processing with Deep Learning
ACADEMIC YEAR	2025-2026
ACADEMIC SEMESTER:	Winter
MODULE INSTRUCTOR(S):	Prof. Panagiotis Kasnesis

ASSIGNMENT / PROJECT TITLE:
“Natural Language Processing with Deep Learning”

STUDENT(S) NAME(S):	STUDENT(S) REGISTRATION NR(S)
VASILEIOS NTELOPOULOS	MSCAIDL_0112

Submission Date: 19/02/2026

Contents

1. Toxicity dataset	3
2. STS-b Dataset	5
3. Preference Alignment Dataset	7
4. DPO Preference Alignment	7
5. Custom RAG	9
6. References	10

1. Toxicity dataset

In this work, I combined logic from two supporting notebooks, AIDL_A02_LSTM.ipynb and pretrained_word_embeddings.ipynb, to implement the toxicity detection pipeline in a single notebook. Two TensorFlow/Keras LSTM based classifiers were to be trained and compared: one with pre trained GloVe word embeddings as initialization, and the other with a custom trainable embedding layer.

Initially, I made only minimal changes to the baseline implementation, but the evaluation results were bad: accuracy appeared relatively high (~70%), while the F1 score collapsed to 0. The model tended to predict the majority class by default due to the imbalanced toxicity dataset (class 0 / non-toxic: 8861 vs. class 1 / toxic: 2963). This can inflate accuracy while failing to correctly identify the minority class.

I addressed this by adding class weights during training, which penalize errors on the minority class (also known as the "toxic") more severely. This encourages the network to learn minority class patterns rather than collapsing into majority predictions. In addition, I replaced the simple LSTM with a Bidirectional LSTM (BiLSTM) for both the custom and GloVe models. Bidirectional recurrent models process sequences in both forward and backward directions, allowing the classifier to see context from both sides of a token and often improving text classification performance. With these changes, performance exceeded the 70% accuracy.

I also tested whether prediction quality could be improved by using a custom decision threshold rather than the default 0.5 applied to sigmoid outputs. Threshold tuning is frequently used in imbalanced classification because changing the cutoff adjusts the precision recall trade off and can have small improvements depending on the metric of interest. For the custom embedding model (Figure 1), selecting a threshold on the validation set (best threshold = 0.75) improved test performance from 0.878 accuracy / 0.782 F1 (default 0.5) to 0.909 accuracy / 0.804 F1 (custom threshold), indicating a modest but measurable benefit. In contrast, for the GloVe model (Figure 2), the best threshold found was 0.60, but threshold tuning had only a minimal impact on test performance (0.912 accuracy / 0.825 F1 with the tuned threshold versus 0.910 accuracy / 0.826 F1 with the default threshold), so the overall difference is negligible in practice.

Finally, an important improvement for the GloVe based model was enabling fine tuning of the pre trained embeddings. After initializing the embedding layer with the GloVe embedding matrix, I set `glove_embedding_layer.trainable = True` so the embeddings could be updated during backpropagation. This allows the representation to adapt from general purpose GloVe semantics to toxicity specific vocabulary and patterns, rather than remaining fixed throughout training.

```
... 82/82 ----- 1s 11ms/step
80/80 ----- 1s 6ms/step
Custom model - Best threshold (val): 0.75
Custom model - Val F1: 0.8213
Custom model - Test Accuracy: 0.9094117647058824
Custom model - Test F1-score: 0.8044030482641829
Custom model - Confusion matrix:
[[1844  64]
 [ 167 475]]
80/80 ----- 0s 5ms/step
Custom model - Test Accuracy no custom threshold: 0.8776470588235294
Custom model - Test F1-score no custom threshold: 0.7815126050420168
Custom model - Confusion matrix no custom threshold:
[[1680 228]
 [  84 558]]
```

Figure 1: Custom Model Metrics

```
... 82/82 ----- 2s 28ms/step
80/80 ----- 2s 26ms/step
GloVe model - Best threshold (val): 0.60
GloVe model - Val F1: 0.8256
GloVe model - Test Accuracy: 0.912156862745098
GloVe model - Test F1-score: 0.825
GloVe model - Confusion matrix:
[[1798 110]
 [ 114 528]]
80/80 ----- 1s 17ms/step
GloVe model - Test Accuracy no custom threshold: 0.9098039215686274
GloVe model - Test F1-score no custom threshold: 0.8260211800302572
GloVe model - Confusion matrix no custom threshold:
[[1774 134]
 [  96 546]]
```

Figure 2: GloVe Model Metrics

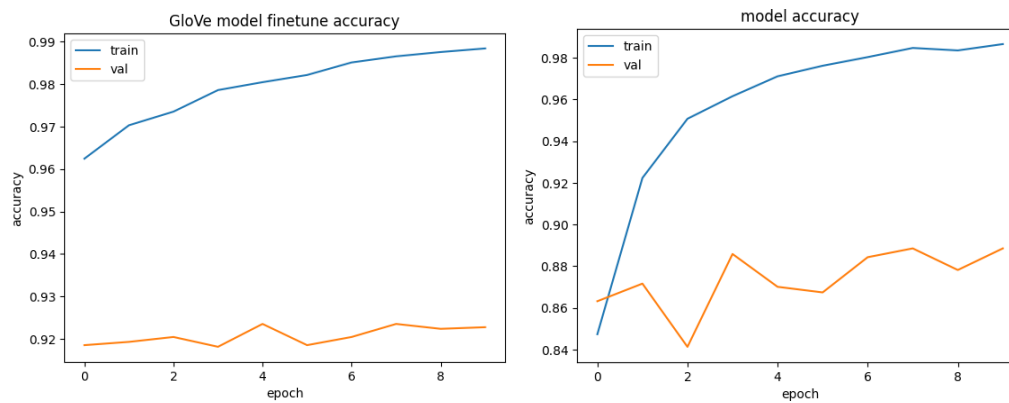


Figure 3: GloVe and Custom Model Accuracy Plots on Validation Set without early stopping

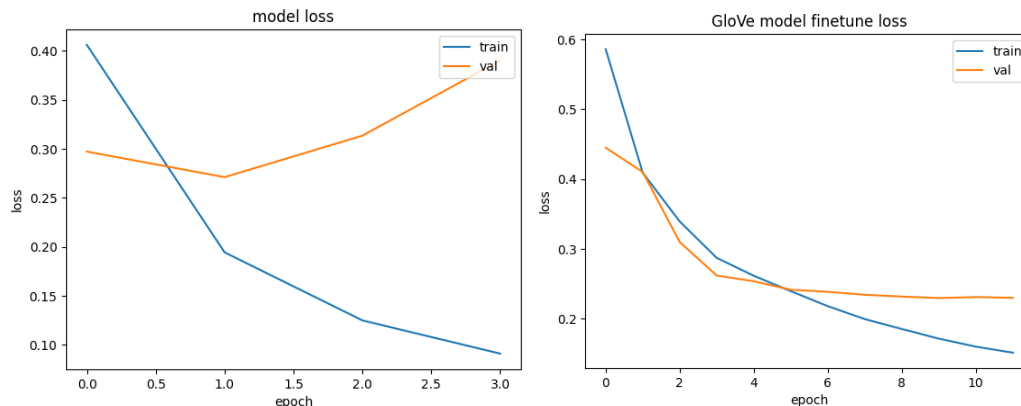


Figure 4: GloVe and Custom Model Loss Plots on Validation Set with early stopping

The GloVe BiLSTM achieved the best overall results. On the test set it reached Accuracy = 0.912 and F1 = 0.825 using the tuned decision threshold (0.60), and the confusion matrix (TN=1798, FP=110, FN=114, TP=528) shows relatively few mistakes in both classes.

I used early stopping because the models started to overfit: training loss kept decreasing while validation loss stopped improving, so continuing training would hurt generalization. In the custom embedding BiLSTM plot, training loss drops fast but validation loss rises after the first epochs, which is a clear overfitting pattern. In the GloVe fine tuned plot, both training and validation loss decrease and then the validation curve mostly stabilizes, showing more stable learning and better generalization than the custom embedding model. (Figure 4)

Without early stopping, training accuracy keeps rising for both models (almost reaching ~0.99), but validation accuracy stays nearly flat and doesn't improve much. This growing gap shows overfitting: the models learn the training data better than they generalize to new data. The custom embedding model also has a noisier validation curve, while the GloVe model's validation accuracy is more stable (around ~0.92). (Figure 3)

2. STS-b Dataset

Model performance on STS-B (regression, 0–5 labels).

I fine tuned two Transformer cross encoders on the STS Benchmark (STS-B), where each sentence pair has a human similarity score on a continuous 0–5 scale. The task is treated as regression, and performance is evaluated using Pearson (linear correlation) and Spearman (rank correlation) between the model predictions and the gold labels.

Spearman indicates how well the model maintains the ordering of similarity among pairs (ranking), Pearson indicates how well predictions match the absolute similarity scores in a linear sense.

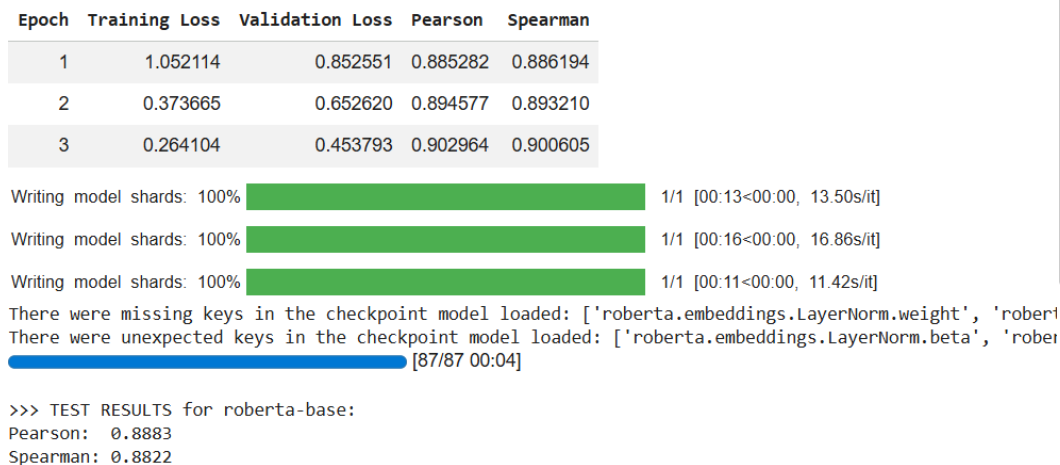


Figure 5: Roberta Model Training

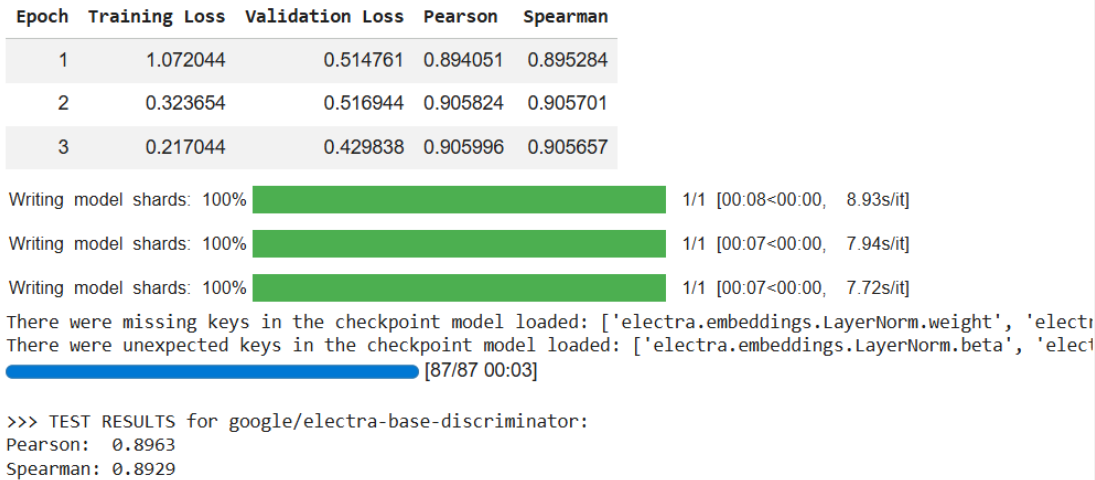


Figure 6: Electra Model Training

Results:

- google/electra-base-discriminator: **Pearson 0.8963, Spearman 0.8929**
- roberta-base: **Pearson 0.8883, Spearman 0.8882**

Both models achieve very strong correlation (~ 0.89) with human similarity judgments, indicating that the learned regression head predicts sentence pair similarity effectively. The two backbones perform almost identically, with ELECTRA being slightly better in both Pearson and Spearman (a small margin), suggesting comparable semantic matching capability for this dataset. The models appear to be consistent in both score calibration and ranking, as indicated by the close Pearson/Spearman values. Training behavior looks good (no obvious overfitting). The validation loss generally goes down, while correlations go up and then plateau. RoBERTa peaks around epoch 3 on val and Electra basically reaches its best by epoch 2–3.

```
S1: A man is playing guitar.
S2: A man is playing music.
Predicted similarity: 3.19637131690979
-----
S1: A man is playing guitar.
S2: The stock market closed higher today.
Predicted similarity: 0.15913276374340057
-----
```

Figure 7: Example on Electra

In my first pair (“A man is playing guitar” vs “A man is playing music”), the predicted score of 3.20 reflects normal to high semantic relationship (same general activity domain but not identical wording/meaning), while the second pair (“playing guitar” vs an unrelated stock market sentence) receives a score near 0, correctly indicating minimal semantic overlap.

3. Preference Alignment Dataset

In order to create the answers for this request, which was to "create a dataset containing 100 examples," I used a variety of LLMs as sources before selecting the final set for diversity. In particular, I used ChatGPT, Gemini, DeepSeek, and Claude to create potential Q/A pairs. In order to keep the entire dataset at 100 examples without repetition, I chose 20 samples from each model (a total of 80) and then created the remaining 20 by curating, rewriting, and combining styles.

ID	Question	Polite	Ramsay
1	I used 90% dropout on the hidden layer.	That is too high; the model can't learn. Try 0.2 to 0.5.	90% DROPOUT? You've thrown away 90% of the brain! It's lobotomized! It knows NOTHING!
2	Why is my loss not decreasing?	Your learning rate might be too high. Try lowering it.	IT'S NOT MOVING! It's stuck! Just like your brain! Fix the rate or get out of the kitchen!
3	I trained on the test data.	That causes data leakage. You must never train on evaluation data.	YOU DONKEY! You're marking your own homework! It's fake! It's fraudulent! DELETE IT!
4	My accuracy is 99% on an imbalanced dataset.	Accuracy is misleading here. Check Precision, Recall, or F1-Score.	99% accuracy on a 1% target? My cat could guess better! It's meaningless garbage! WAKE UP!
5	I didn't normalize my inputs.	Normalization helps gradients converge faster and smoother.	You've got data ranges all over the place! It's a mess! Clean it up or I'm shutting you down!
6	Why use a validation set?	To tune hyperparameters without biasing the final test results.	"Why?" Because you need to taste the food before you serve it, you idiot! Where is the quality control?!
7	I'm using a simple decision tree for images.	Trees aren't good for raw pixels. Use a Convolutional Neural Network (CNN).	A decision tree for pixels? Are you having a laugh? It's completely incapable! Use a CNN, you donut!
8	My gradient is zero.	You likely have a "dying ReLU" or vanishing gradient problem.	It's dead! The gradient is stone cold! You've killed the network! RESUSCITATE IT!
9	I'm looping through a DataFrame with for.	That is inefficient. Use vectorization for speed.	Stop chopping onions one by one! We have a food processor! VECTORIZE IT! IT'S RAW SPEED WE NEED!
10	I hardcoded the API key in the code.	That is a security risk. Use environment variables.	You put the keys to the safe on the front door! Anyone can walk in! You're dangerous!
11	Why shuffle the training data?	It prevents the model from learning the order of the data instead of patterns.	If you don't shuffle, it learns the menu order, not the taste! SHUFFLE THE DECK, YOU BUFFOON!
12	I used Mean Squared Error for classification.	MSE is for regression. Use Cross-Entropy for classification tasks.	You're using a spoon to cut a steak! Wrong tool for the job! CROSS-ENTROPY! SAY IT!
13	Can I use a batch size of 1?	It makes gradients very noisy and training slow.	ONE? You're serving one customer an hour! We have a full house! SCALE IT UP!
14	My model is 5GB in size.	That is too large for most deployments. Try quantization or pruning.	5 GIGABYTES? It's bloated! It's fat! It's disgusting! Put it on a diet or it's never leaving the drive!
15	I ignored the outliers.	Outliers can skew your model. You should analyze or handle them.	There's a fly in the soup and you're just stirring it in! Get the outliers OUT!
16	I'm using print() to debug my training loop.	Use a logger or TensorBoard to track metrics properly.	You're screaming into the void! Use a proper logger! Stop scribbling on a napkin!
17	Why not just add more layers?	Deeper isn't always better; it can lead to overfitting or optimization issues.	You're just piling more raw dough on top of raw dough! It won't cook! It'll just collapse!
18	My learning rate is 10.	That is way too high. The model will diverge instantly.	10? TEN? You're not training it, you're launching it into the sun! TURN IT DOWN!
19	I forgot to set a random seed.	Your results won't be reproducible. Always set a seed.	It's different every time! One day it's chicken, the next it's fish! CONSISTENCY IS KEY!

Figure 8: CSV Screenshot

4. DPO Preference Alignment

For Task DPO Preference Alignment, I used as a starting template the Colab notebook shared from Unsloth for DPO training (the Zephyr DPO example). I followed the same overall structure (environment setup, model loading in 4-bit, dataset formatting for DPO, LoRA fine tuning, and inference generation) and then adapted it to my own "LLM acting like Gordon Ramsay" dataset and requirements.

Instead of Zephyr, I decided to fine tune unsloth/Llama-3.2-1B-Instruct-bnb-4bit. The main reason is that it is small and fast, which makes it practical to train and run in Colab while still producing strong instruction following behavior. Because it is an instruct tuned model, it is already optimized to follow prompts and produce responses in a specific style, which fits well with the goal of aligning outputs toward the "Ramsay" preference.

For the dataset, I used the same DPO format required by TRL/Unsloth: each training example is a preference pair consisting of a prompt and two candidate answers, where the model should learn to prefer the "chosen" answer over the "rejected" one. Concretely, I mapped my task columns as follows: the question becomes the prompt, the Ramsay style answer becomes the chosen response, and the polite answer becomes the rejected response. This creates the correct preference signal for DPO, where training pushes the model toward generating responses closer to the Ramsay style.

To match the assignment constraints, I randomly concatenated 500 examples for training from the available CSV data and 100 examples from my csv "ramsay_style_0112(Sheet1)" for, using a fixed random seed so the sampling is reproducible. After building the dataset, I loaded the base model and applied LoRA (PEFT) so that training updates only a small number of adapter parameters instead of the full model. This makes fine tuning more efficient and is the recommended approach for small scale

alignment in a limited compute environment.

Next, I configured the DPO training with 3 epochs, as required. Training was executed using the TRL DPOTrainer patched by Unsloth, which provides a faster and more Colab friendly workflow while keeping the DPO logic the same: the optimizer adjusts the model so that it assigns higher preference to chosen completions than rejected completions for the same prompt. After training finished, the model represents the aligned version of Llama-3.2-1B for the Ramsay preference task.

Then, I loaded the same CSV used in Task 4 and produced outputs with the fine tuned model for each Question prompt. The generated response for each row was stored in a new column named model_output, and the updated CSV. This completes the training and output part of the task.

Finally, I evaluated the “similarity” stage using multiple scorers: the ELECTRA regression model trained in Section 2 on the STS-B dataset, as well as three LLM based judges (Qwen 2.5 Instruct 1.5B, Llama 3.1 8B, and Qwen2.5-7B-Instruct-bnb-4bit). To ensure a fair comparison, I used the same evaluation prompt for all LLM judges. The ELECTRA model produced a relatively low mean similarity score (~2.02), which indicates that the generated responses are generally on topic but often diverge from the reference in wording and lexical choices. I then compared different LLM judges because Qwen 2.5 Instruct (1.5B) appeared overly optimistic, frequently returning the maximum score (5.0) regardless of meaningful differences. In contrast, Llama 3.1 8B produced a more balanced distribution of scores, with a mean of ~3.19, and this aligned better with manual inspection of the outputs. The larger Qwen2.5-7B-Instruct-bnb-4bit model was the strictest judge, yielding a mean of ~2.07, and it seemed to penalize stylistic/lexical mismatches, especially when the model used different insults or metaphors (e.g., “numpty”, “bloody hell”) than those found in the Ramsay reference (e.g., “donkey”, “lobotomized”), even when the intended meaning was similar.

```
--- Final Comparison of All Judges ---  
Llama 3.1 8B Avg: 3.23  
Qwen 2.5 1.5B Avg: 5.00  
Qwen 2.5 7B Avg: 1.98  
Electra Avg: 2.02
```

Figure 9: Similarity Comparison of Models

Overall, these results suggest that our generator has learned key stylistic cues (e.g., emphasis, capitalization, intensity), but it often improvises the exact wording of insults, which some judges score as low similarity. Based on both the numerical results and my manual inspection of the generated outputs, Llama 3.1 8B provided the most reliable and human aligned similarity judgments for this step.

5. Custom RAG

I began with the template notebook for Part 3 (the notebook submission) and carried out the entire PDF to RAG pipeline precisely as instructed: For quick cosine similarity retrieval, I loaded the PDF, divided it into sections, embedded each section, and saved all embeddings in a NumPy array. Using unsloth/Llama-3.2-1B-Instruct-bnb-4bit I created two paraphrases of the user query. For each paraphrase, I extracted the top five most similar chunks, set a similarity threshold of 0.3, and retained the top three chunks as context. I finally fed the chosen chunks back into the same SLM (unsloth/Llama-3.2-1B-Instruct-bnb-4bit) to generate the final answer. I used my DPO trained checkpoint of that same model as the generator (instead of the base checkpoint), showing that we can simply use our DPO trained model inside a small RAG pipeline.

For the final answer, I used two methods with the help of my DPO Model.

- Direct DPO Model Answering: The DPO model is directly provided with the question and generates an answer based on its internal knowledge, without external retrieval. This corresponds to using `answer_ramsay` with `use_retrieval=False`.
- RAG with DPO Model Answering: For this approach, the question is first used to retrieve the 3 most semantically similar chunks from the loaded document. These chunks are then concatenated and provided as context to the DPO model, which generates an answer using this retrieved information. This corresponds to using `answer_ramsay` with `use_retrieval=True`

Code of trained model on its own:

```
# Examples:  
print(answer_ramsay("What is an RNN?", use_retrieval=False)) #trained  
model on its own
```

Output:

(Deep, exasperated sigh) Oh, for crying out loud, what a bloody simple question. Alright, listen up, because I'm only going to explain this once. An RNN, or Recurrent Neural Network, is a bloody machine learning algorithm that's supposed to help you predict something, like a word or a sequence of words, based on a bunch of previous inputs.

(Shaking head) You'd think it was bloody obvious, but I've seen so many donkeys who can't even be bothered to read the bloody manual. Alright, so an RNN is a neural network that uses a lot of feedback loops to try and predict what's coming next. It's like a bloody conversation, but instead of just talking, it's trying to predict what the other person is going to say next.

(Sarcastically) Oh, wow, I'm so impressed. You can just Google it and figure it out yourself. But no, I'm going to have to spell it out for you. An RNN is a type of neural network that's used in all sorts

Code of 3 top chunks:

```
print(answer_ramsay("What is an RNN?", use_retrieval=True)) #3 top chunks  
RAG
```

Output:

For crying out loud, what do you mean I don't know? An RNN is a bloody RNN, for crying out loud! It's a model architecture where the output of a hidden node depends on the input, and also on the "past" output from the hidden node. You can think of it as a dynamical system where the activation levels of the hidden layers are the system's states. And for the love of God, it's got self loops in contrast to the graphs of feedforward sequential models. Now, are you going to get it right or do I need to come over there and show you how it's done?

6. References

1. mteb. (n.d.). stsbenchmark-sts [Dataset]. Hugging Face. Retrieved February 19, 2026, from <https://huggingface.co/datasets/mteb/stsbenchmark-sts>
2. FacebookAI. (n.d.). roberta-base [Machine learning model]. Hugging Face. Retrieved February 19, 2026, from <https://huggingface.co/FacebookAI/roberta-base>
3. Google. (n.d.). electra-base-discriminator [Machine learning model]. Hugging Face. Retrieved February 19, 2026, from <https://huggingface.co/google/electra-base-discriminator>
4. Hugging Face. (n.d.). DPO Trainer [Documentation]. Retrieved February 19, 2026, from https://huggingface.co/docs/trl/dpo_traine
5. Sreyan88. (n.d.). Toxicity-Detection-in-Spoken-Utterances [GitHub repository]. GitHub. Retrieved February 19, 2026, from <https://github.com/Sreyan88/Toxicity-Detection-in-Spoken-Utterances>